Linux Troubleshooting — Scenario #1: Disk Almost Full

DevOps on-call runbook • Syed Aftab • Reference for real incidents & interviews

> 🚨 **The alert (7:03 AM):** `web01` — root filesystem `/` at **95%**. Application writes may start failing.

A full disk is the **#1 most common production incident**. It stops databases, breaks logging, and fails deploys. This runbook is the exact flow to resolve it.

The methodology (memorize this flow)

#	Phase	Command	Goal
1	Confirm	`df -h /`	Is it real? How bad?
2	Locate	`du -xh / | sort -rh | head`	Follow size DOWN the tree to the culprit
3	Identify	(read the output)	Recognize WHAT it is (log? container? cache?)
4	Fix safely	right tool for that culprit	Reclaim space without corrupting anything
5	Verify	`df -h /` again	Confirm space came back
6	Prevent	cron cleanup + disk alert	Stop the repeat

STEP 1 Confirm — `df -h /`

```
df -h /
```

Part	Meaning
`df`	**d**isk **f**ree — how full is the filesystem
`-h`	human-readable sizes (G, M)
`/`	check only the root filesystem (the one that matters)

Read the **Use%** column. Over ~90% needs action; over 95% is urgent.

> **Red-herring lesson:** not every 100% is an emergency. A read-only system mount (e.g. WSL's `linux-tools` squashfs) is *supposed* to be 100% — it's sealed and can't be written to. Always ask: **which** filesystem, and is it **writable**? The one that fills in real incidents is `/` or a data mount.

STEP 2 Locate — find what's using the space

```
sudo du -xh / 2>/dev/null | sort -rh | head -10
```

Part	Meaning
`du -h`	**d**isk **u**sage of files/folders, human-readable
`-x`	stay on ONE filesystem (don't wander into other mounts)
`/`	start from root
`2>/dev/null`	hide "permission denied" errors — but KEEP the results
`sort -rh`	biggest first (`-r` reverse, `-h` human sizes)
`head -10`	top 10 only

> **Why `2>/dev/null` and NOT `&>/dev/null`:** here the **output is what we want** (the sizes, to pipe into `sort`). `2>` hides only errors and keeps results. `&>` would hide *everything*, leaving `sort` with nothing. Use `&>` only when you don't care about any output (e.g. an SSH reachability check where only the exit code matters).

The core skill: follow the size *down the tree*. The output led us: `/` → `/var` → `/var/lib` → **`/var/lib/containerd` (~7G)**.

STEP 3 Identify — what is the culprit?

`/var/lib/containerd` = the **container runtime**'s data (the engine Docker/Kubernetes use). It holds **container images and their stacked filesystem layers** (`overlayfs/snapshots`).

Usual culprit	What it is
`/var/log`	runaway log files (#1 cause overall)
`/var/lib/containerd` or `/var/lib/docker`	container image / layer buildup (our case)
`/tmp`	temp files never cleaned
`/var/cache/apt`	old package downloads

a user's home	a huge file/backup someone dumped

This was **container buildup**, not a log — which means a different, safe fix (NOT `rm`).

STEP 4 Investigate & fix — the container tools

```
docker system df           # see where container space goes
```

Key column is **RECLAIMABLE** — how much is safe to free. Example: Images 7.3GB total, 5.5GB reclaimable (only 3 of 8 images in use); Build Cache 3.9GB.

```
docker system prune -a     # reclaim unused images + stopped containers + build cache
```

Part	Meaning
`docker system prune`	remove stopped containers, dangling images, unused networks, build cache
`-a`	ALSO remove images not currently used by a container (frees the big chunk)

> **Safety:** `-a` removes images not backing a container *right now* — you simply re-pull them later (a download, not data loss). It does **NOT** touch volumes, so your data is safe. Never add `--volumes` unless you're 100% sure no data lives in them. **Never** `rm` files under `/var/lib/containerd` directly — it corrupts the runtime's internal state.

STEP 5 Verify

```
df -h /
sudo du -xh / 2>/dev/null | sort -rh | head -10
```

> **Result:** `/` dropped **12G → 5.1G**; `/var` went **8.0G → 1.2G**. **~7GB reclaimed.** `containerd` no longer in the top 10. Incident resolved. ✅

STEP 6 Prevent

1. Scheduled cleanup (cron):

```
0 3 * * 0   docker system prune -f     # Sunday 3AM, SAFE (no -a)
```

cron time fields: `minute hour day-of-month month day-of-week`. So `0 3 * * 0` = minute 0, hour 3, any day, any month, Sunday. `-f` alone removes only dangling/unused junk — safe to automate; save `-a` for manual runs.

2. Early warning — alert at 80% so you get days of notice:

```
*/30 * * * *  ~/Linux-Daily/day6_disk_check.sh 80 || alert...
# */30 = every 30 minutes
```

Full command reference

Command	What	Why here
`df -h /`	disk free, root only, human sizes	confirm the disk is really full
`du -xh / | sort -rh | head`	biggest dirs on one filesystem	locate the culprit by size
`2>/dev/null`	discard errors, keep output	silence "permission denied" noise
`docker system df`	container space breakdown	see how much is reclaimable
`docker system prune -a`	remove unused images/containers/cache	reclaim space safely
`crontab -e` / `-l`	edit / list scheduled jobs	automate prevention

Interview answer (say this)

"*Disk full? I **confirm** with `df -h`, then **locate** the culprit with `du` sorted by size, following it down the tree. I **identify** what it is — a runaway log, container images, cache — then use the **right tool** to fix it (e.g. `docker system prune`, never `rm` on runtime data). I **verify** the space returned, then **prevent** a repeat with a cron cleanup and a disk-usage alert at 80%.*"